https://youtu.be/mhwp4Ly4c84?si=xHYIfGt1nSVQUT1u



WezTerm Conceptual Architecture

CISC 322 - Group 11: g(ai)ls

g(ai)ls

- **Akshay Kapal** → Group Leader, Architecture
- **Sophie Liang** → Group Presenter, Introduction, Conclusion
- **Manreet Gill** → Group Presenter, Abstract, References, AI Collaboration Report
- **Ava Nikolaus** → Architecture
- **Aimee Anwar** → Derivation Process, External Interfaces
- **Maria Koukarsky** → Use Cases, Data Dictionary, Naming Conventions

What is WezTerm

- WezTerm is a cross-platform terminal emulator and multiplexer built in Rust.
- Terminal emulator → runs your shell and draws its output, handling colours, cursor movement, and other escape codes
- Multiplexer → manages windows, tabs, and split panes itself, so sessions can keep running locally or on a remote machine
- GPU rendering → text is drawn on the GPU using cached glyphs, so scrolling stays smooth
- Lua config → settings are a Lua script, so key bindings, themes, and behaviour can be customized with code



Data Dictionary

- **Client-server:** A GUI client connects to a mux server that holds the terminal state, so sessions can outlive a disconnected client and reconnect.
- **ConPTY:** The Windows pseudo-console, which plays the role of the Unix PTY.
- **Domain:** A group of windows, tabs and panes tied to a place where they run: the local machine or a remote multiplexer reached over SSH, a Unix socket or TLS.
- **Escape sequence:** Special bytes in program output that tell the terminal how to draw, such as changing colour or moving the cursor. WezTerm also encodes keyboard and mouse input as escape sequences for the program.
- **Event-driven (implicit invocation):** Components react to named events instead of being called directly. Key presses are dispatched to actions, and Lua handlers respond to events such as gui-startup.
- **Key binding:** A rule, from the defaults or the user's Lua file, that maps a key combination to a named action such as splitting a pane.
- **Lua configuration:** The Lua script that configures WezTerm ("code-as-configuration"), loaded at startup and reloaded when it changes.
- **Lua event handler:** A Lua function registered with wezterm.on that runs when a named event occurs, such as gui-startup or mux-startup.
- **Mux server:** A multiplexer daemon that can run without a GUI and keeps terminal state while clients connect and disconnect.
- **Observer pattern:** The Multiplexer sends notifications, such as a new pane being created, to components that subscribe to them.

Data Dictionary (cont.)

- **OSC (Operating System Command):** Escape sequences used for shell integration, such as OSC 7 (working directory) and OSC 133 (prompt, input and output zones).
- **Pane, tab and window:** A window holds tabs and a tab holds panes. A pane is a view of one independent terminal session, tracking its PTY process and screen content.
- **Pane threads:** Each pane has a reader thread that does blocking reads from the PTY and a parser thread that interprets the escape sequences.
- **PTY (pseudo-terminal):** A channel between userspace and the kernel. The controller side carries information such as window size, and the client side is the I/O stream used by the program.
- **Screen state and scrollback:** The terminal's current contents (characters, colours, cursor) and the saved lines that have scrolled off.
- **Shell:** A command interpreter such as bash or zsh, run inside a pane. Not part of WezTerm.
- **Terminal emulator:** A program that starts a shell in a pseudo-terminal, interprets the escape sequences it emits to draw a display, and turns input into escape sequences for it.
- **Transport:** The connection type between WezTerm and a remote session: SSH, Unix socket or TLS. Serial ports are also supported as a client connection.

Naming Conventions

- Abbreviations:
  - **API:** Application Programming Interface
  - **CLI:** Command Line Interface
  - **GPU:** Graphics Processing Unit
  - **GUI:** Graphical User Interface
  - **UI:** User Interface
  - **OS:** Operating System
  - **PTY:** Pseudo-Terminal
  - **ConPTY:** Windows Pseudo Console
  - **Mux:** Multiplexer
  - **SSH:** Secure Shell
  - **TLS:** Transport Layer Security

Architecture Styles

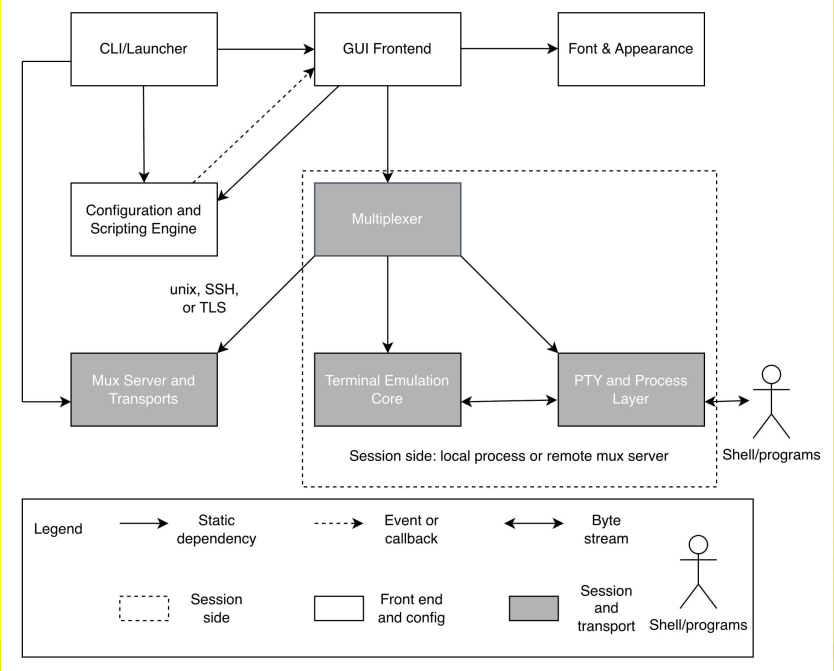
Client-Server Style

- GUI client connects to multiplexer server over Unix socket, SSH, or TLS
- Server keeps terminal state (even color palette for remote domains)
- Sessions survive disconnects and resume automatically (TLS)
- Presence of latency so WezTerm adds predictive local echo
- Compatible WezTerm version required on remote host, plus certificate and socket security setup.

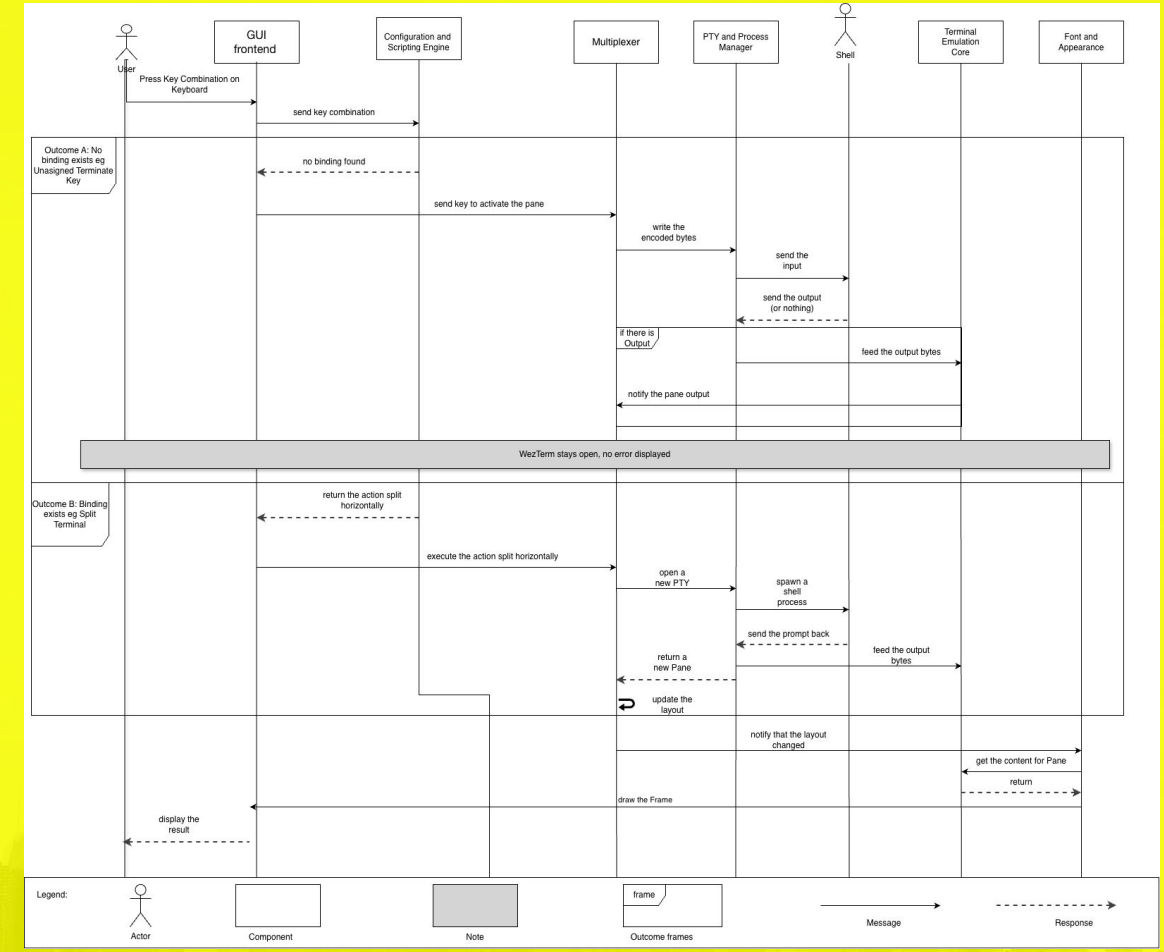
Event Driven

- Key presses dispatch to actions, Lua handlers subscribe to named events (wezterm.on)
- Events fire from both GUI and mux layers (gui-startup, mux-startup)
- Users extend WezTerm without changing it (custom events, plugin, hot reload)
- Handler order is easy to get wrong, and a handler returning false can suppress others
- Async operations can't be awaited, and config reloads can stack up side effects

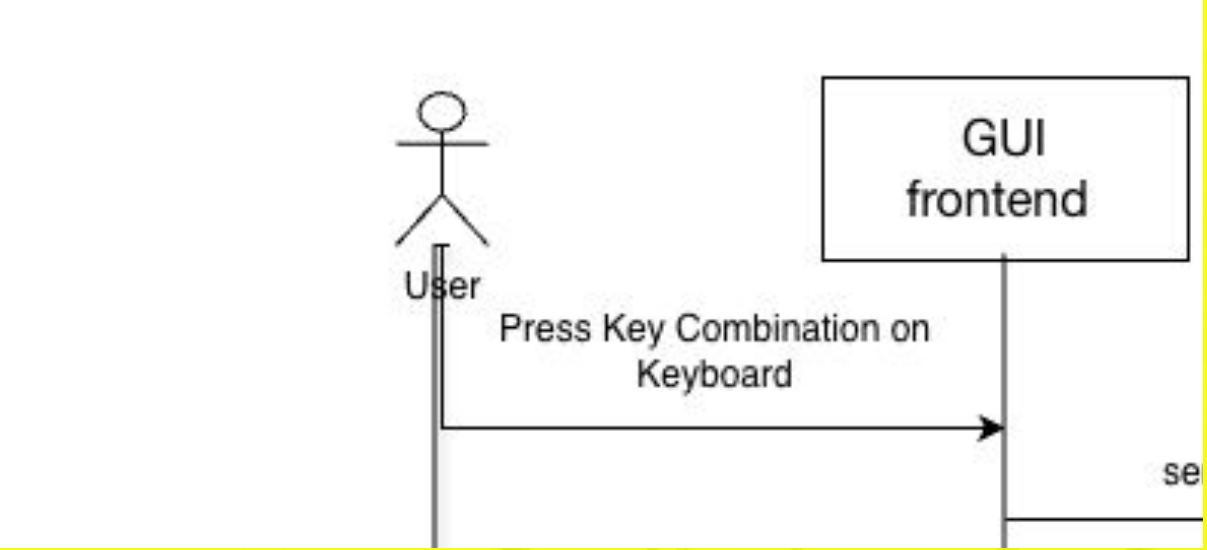
Box-and-Arrow Diagram



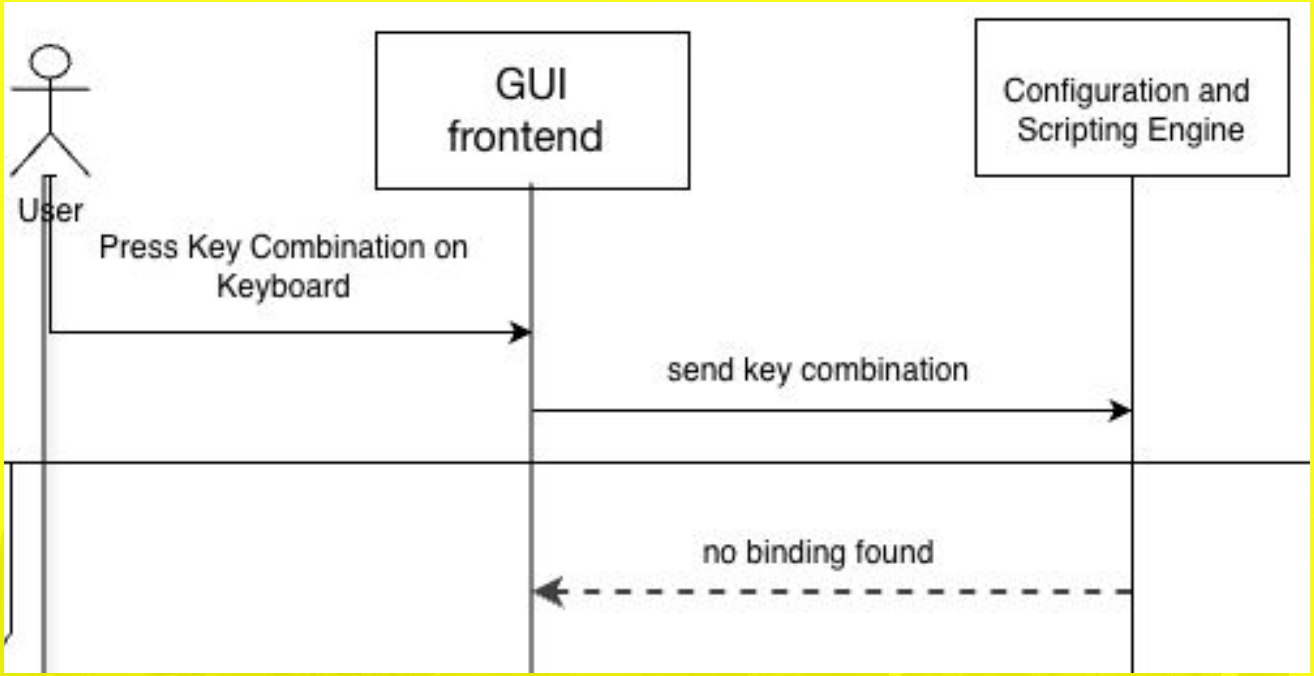
USE CASE 1


User
GUI frontend
Configuration and Scripting Engine
Multiplexer
PTY and Process Manager
Shell
Terminal Emulation Core
Font and Appearance
Press Key Combination on Keyboard
send key combination
Outcome A: No binding exists eg Unasigned Terminate Key
no binding found
send key to activate the pane
write the encoded bytes
send the input
send the output (or nothing)
if there is Output
feed the output bytes
notify the pane output
WezTerm stays open, no error displayed
Outcome B: Binding exists eg Split Terminal
return the action split horizontally
execute the action split horizontally
open a new PTY
spawn a shell process
send the prompt back
feed the output bytes
return a new Pane
update the layout
notify that the layout changed
get the content for Pane
return
draw the Frame
display the result
Legend:
Actor
Component
Note
frame
Outcome frames
Message
Response

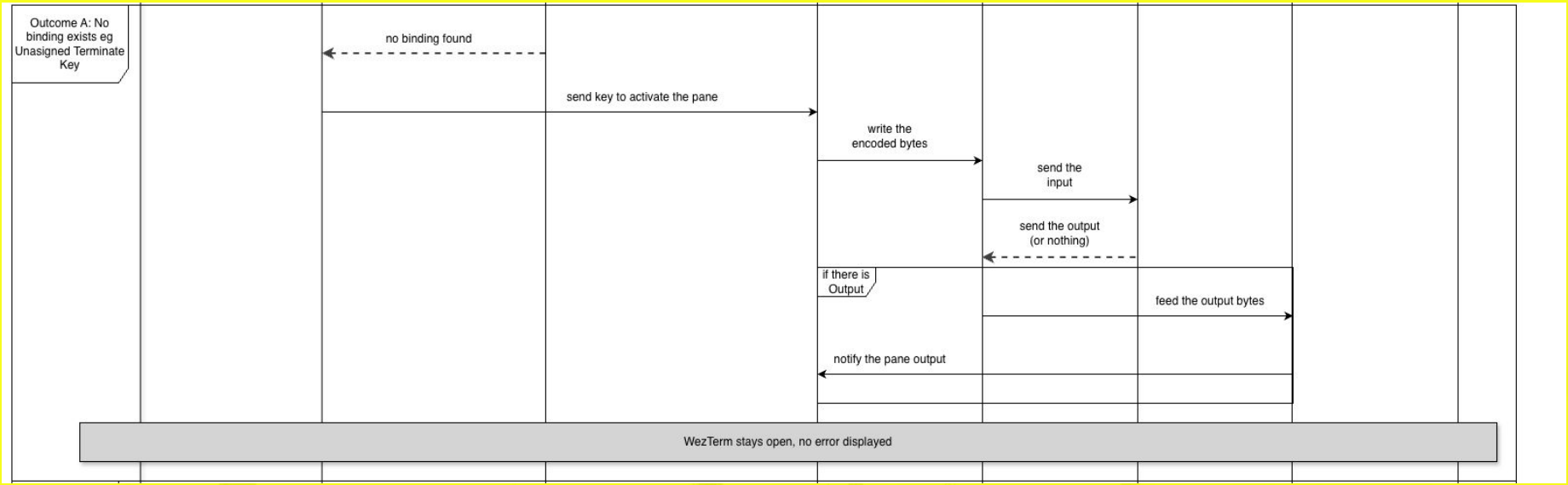
Phase 1



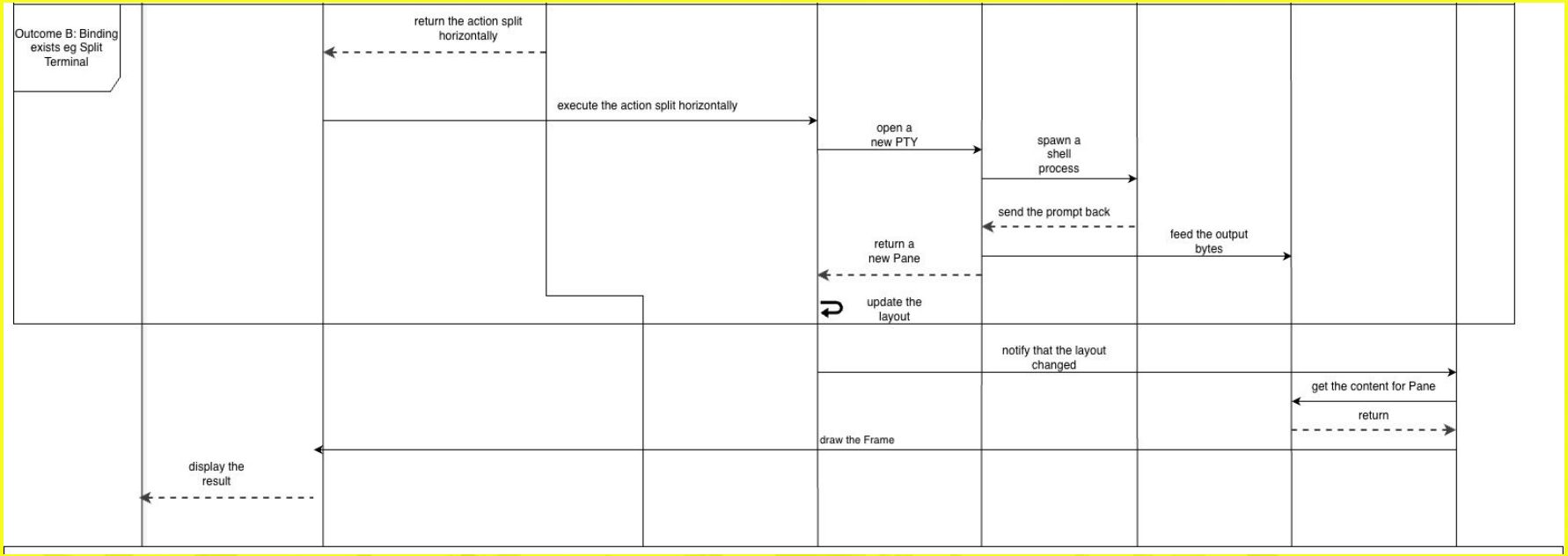
Phase 2


GUI
frontend
Configuration and
Scripting Engine
User
Press Key Combination on
Keyboard
send key combination
no binding found

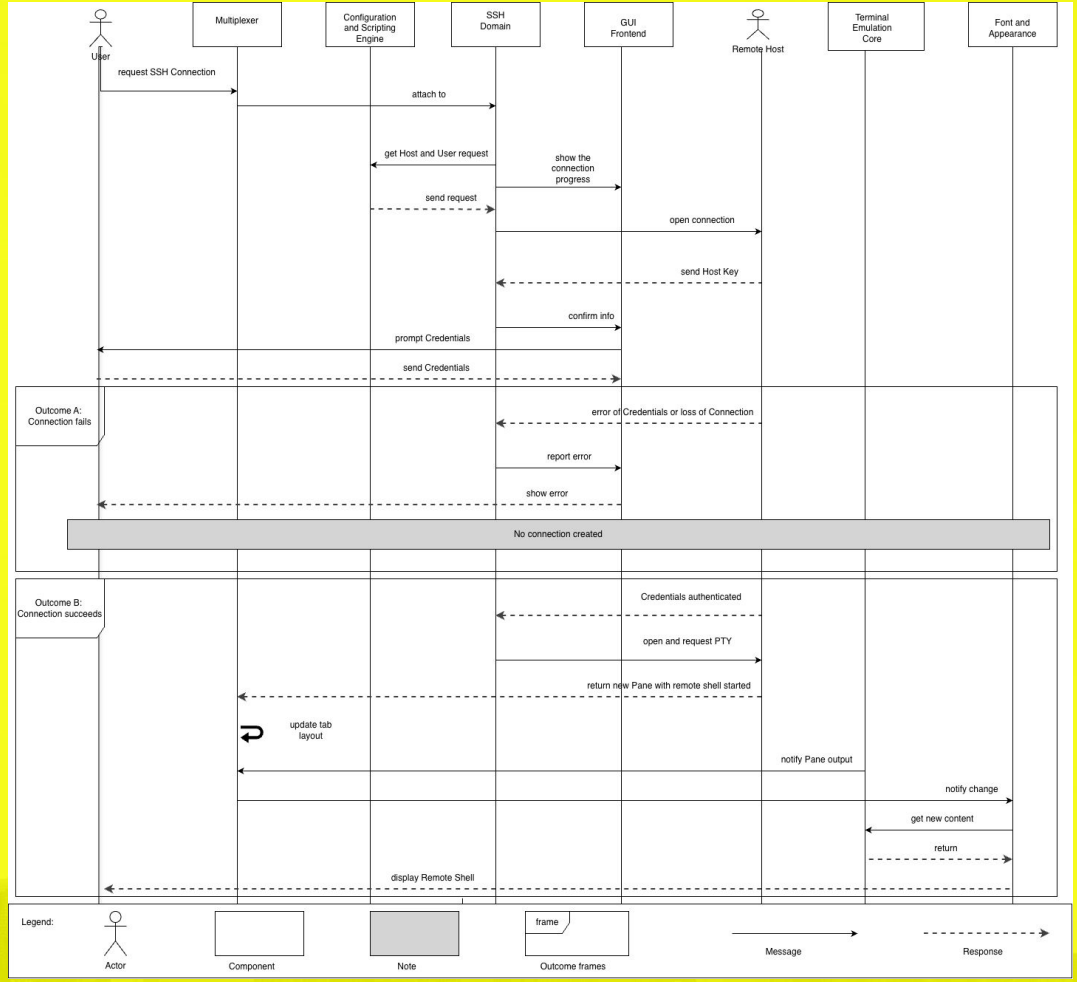
Outcome A


Outcome A: No binding exists eg Unasigned Terminate Key
no binding found
send key to activate the pane
write the encoded bytes
send the input
send the output (or nothing)
if there is Output
feed the output bytes
notify the pane output
WezTerm stays open, no error displayed

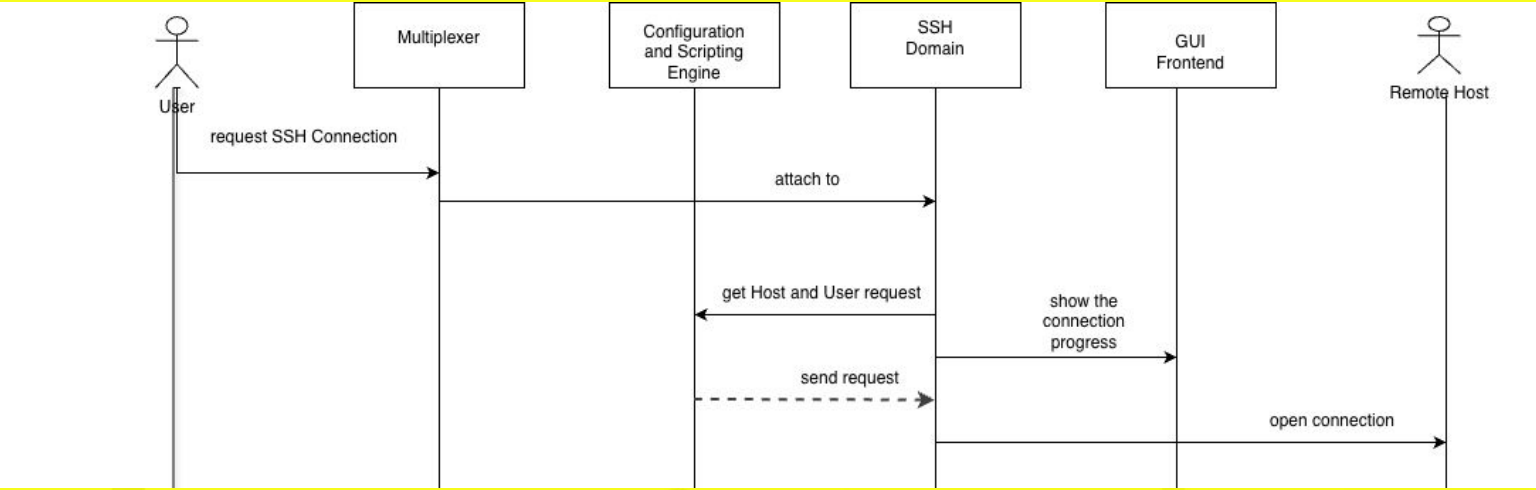
Outcome B


Outcome B: Binding exists eg Split Terminal
return the action split horizontally
execute the action split horizontally
open a new PTY
spawn a shell process
send the prompt back
feed the output bytes
return a new Pane
update the layout
notify that the layout changed
get the content for Pane
return
draw the Frame
display the result

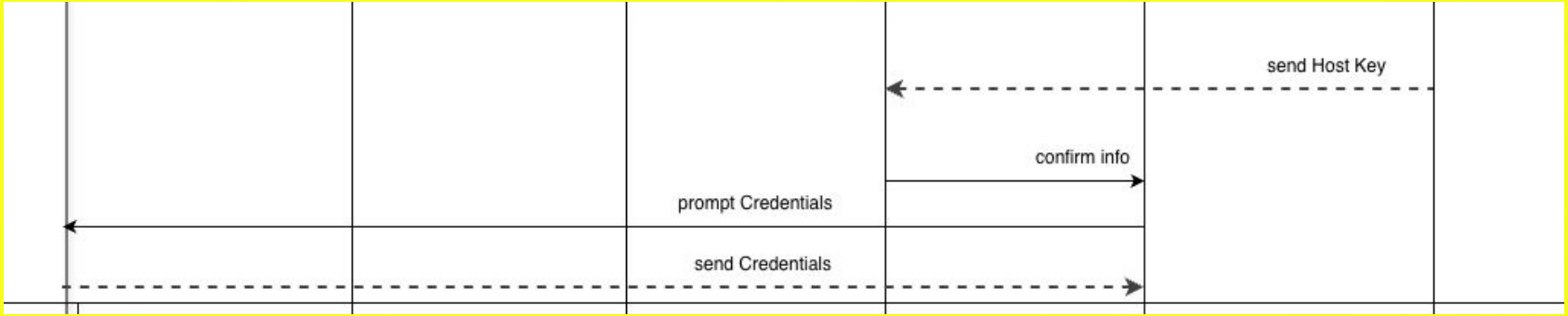
USE CASE 2


User
Multiplexer
Configuration and Scripting Engine
SSH Domain
GUI Frontend
Remote Host
Terminal Emulation Core
Font and Appearance
request SSH Connection
attach to
get Host and User request
show the connection progress
send request
open connection
send Host Key
confirm info
prompt Credentials
send Credentials
Outcome A: Connection fails
error or Credentials or loss of Connection
report error
show error
No connection created
Outcome B: Connection succeeds
Credentials authenticated
open and request PTY
return new Pane with remote shell started
update tab layout
notify Pane output
notify change
get new content
return
display Remote Shell
Legend:
Actor
Component
Note
frame
Outcome frames
Message
Response

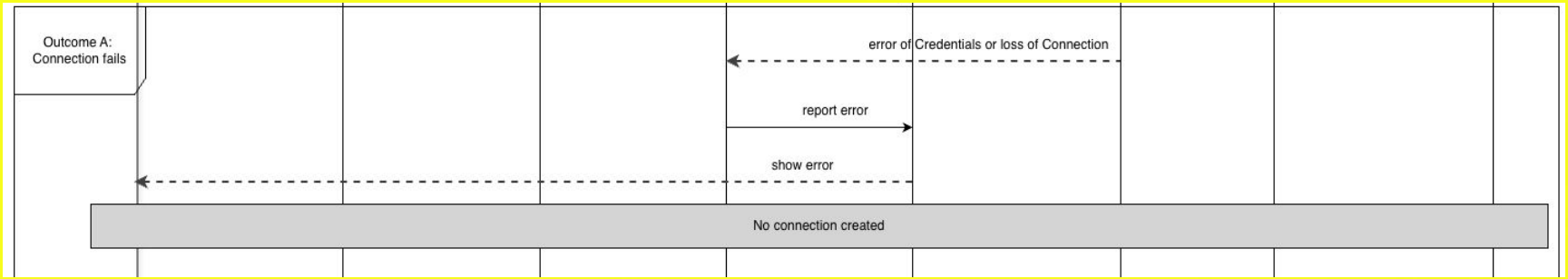
Phase 1


User
Multiplexer
Configuration and Scripting Engine
SSH Domain
GUI Frontend
Remote Host
request SSH Connection
attach to
get Host and User request
show the connection progress
send request
open connection

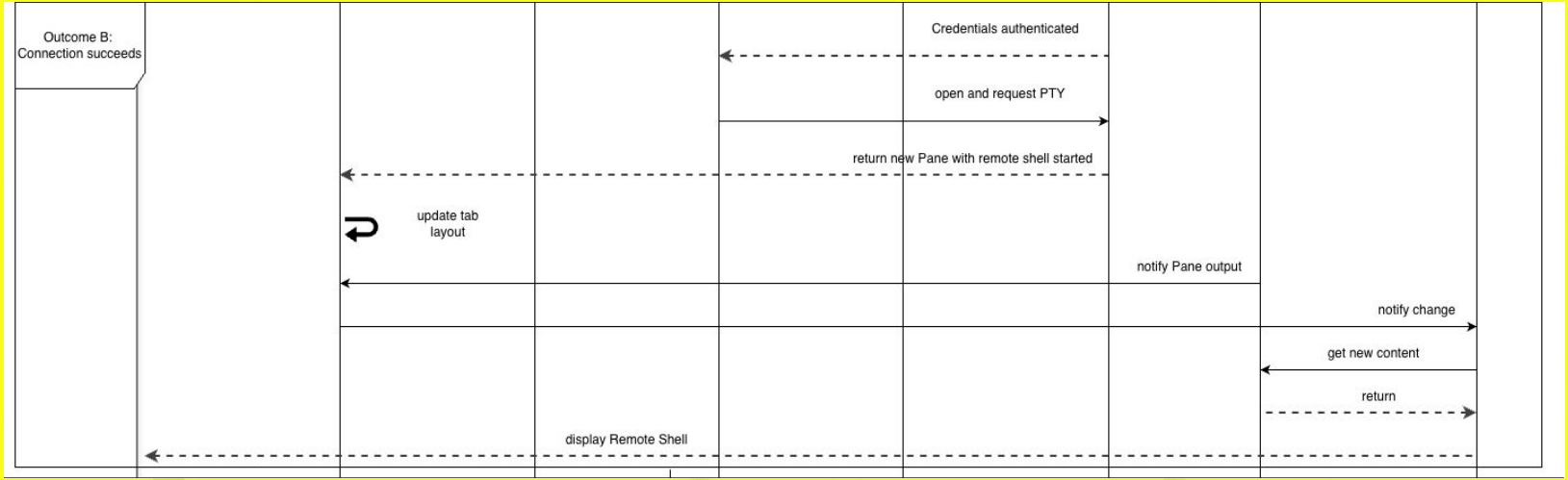
Phase 2


send Host Key
confirm info
prompt Credentials
send Credentials

Outcome A


Outcome A:
Connection fails
error of Credentials or loss of Connection
report error
show error
No connection created

Outcome B


Outcome B:
Connection succeeds
Credentials authenticated
open and request PTY
return new Pane with remote shell started
update tab
layout
notify Pane output
notify change
get new content
return
display Remote Shell

Control Flow & Concurrency

1. **Startup**

➔ The main thread is marked before calling run, the program parses the command line and sets up environment then determines the subcommand (start, connect, ssh, serial, ls-fonts, show-keys).

2. **Configuration**

➔ The first file found in the config path runs synchronously and returns a validated table of settings. If the file is broken the current setting are kept and the error is stored, otherwise the changes are applied.

3. **Window Creation**

➔ When the gui-startup event is triggered a multiplexer window is built and notifies the frontend to create a matching GUI window. The window determines its size based on its tab and connect to the display to begin running.

4. **Input Event**

➔ Events go through one function on the GUI thread. When a key press occurs the window verifies if it is the leader key, if an active interface wants it and if it key binding. The leader key also begins a short timer for which only key bindings are accepted.

5. **Key Binding**

➔ If a binding matches, its action runs otherwise the key goes to the active pane. There are two handlers processing keys one matching bindings and the other cross-referencing window and default tables.

6. **Pane Output & Exit**

➔ Each pane has two background threads a reader that performs blocking reads and a parser the analyzes escapes sequences, groups output and adds the results. The main thread is then notified to check if the pane is visible. Finally, the paint routine runs drawing the frames details (background, borders etc.) before exiting.

Interactions Between Subsystems

Process Launch: CLI Launcher → GUI Frontend
Config Selection: CLI Launcher → Configuration
Connection Setup: CLI Launcher → Mux Server and Transports
Settings Loading: GUI Frontend → Configuration
Scripting Callbacks: Configuration → GUI Frontend
Rendering Requests: GUI Frontend → Font and Appearance
Domain Attachment: GUI Frontend → Multiplexer
Pane State Management: Multiplexer → Terminal Emulation Core
Process Spawning: Multiplexer → PTY and Process Layer
Remote Domains: Multiplexer → Mux Server and Transports
I/O Streaming: Terminal Emulation Core → PTY and Process Layer

Derivation Process

- Configured using eight components
  - CLI/Launcher
  - GUI frontend
  - Configuration and scripting engine
  - Font and appearance
  - Terminal emulation core
  - Multiplexer
  - Multiplexer servers and transports
  - PTY and Process Layer
- Focused on streamlining the terminal environment's user interface and allowing flexibility in the configuration of the terminal

Alternative Architectures

Layered

The system is divided into distinct layers, each with a specific function, communicating with neighbouring layers.

Flow: User input passes through each layer processing what the above layer passes down.

Advantages:

- Improved organization and management since each layer had a defined task.
- Components can be modified without impacting the rest of the system.

Disadvantages:

- Concurrency is limited by communication between layers.
- A change request must pass through multiple layers causing delay.

Pipe-and-Filter

The system is a group of independent transformations that are handled by different components.

Flow: A program in a pane would produce output via PTY which is passed to the parser to analyze and update state before displaying it.

Advantages:

- Supports concurrency by allowing different processing steps run simultaneously.
- Allows flexibility as the parser can be changed without impacting the PTY or display.

Disadvantages:

- The shift from the event-driven style could add overhead as data passes between stages.
- A result of Indirect communication through pipes versus direct response to events.

Lessons Learnt

- **Use Cases are Goals**
  - We first counted each outcome as its own use case
  - One use case covers the success path and everything that can go wrong or branch (shown as outcome frames)
  - Even a simple interaction has many failure points: unbound key, rejected host key, refused login, unreachable host

- **Names and Plans Come First**
  - We ended up with three sets of names: box diagram, sequence diagrams, and naming conventions
  - Reconciling them cost time and risked consistency marks
  - Sections depended on the final box diagram, so placeholder names caused rework

Lessons Learnt

- **Humans Must Stay in the Loop**
  - The AI saved time on drafts and reviews, but sometimes sounded sure about details it hadn't verified, and its first component list came from memory
  - Every claim about WezTerm was checked against the source, docs, or course material
  - The AI helped us think, but the group had to decide

- **A Plan With Clear Ownership Prevents Rework**
  - Every section had an owner, but use cases and component descriptions both depended on the final box diagram
  - Without a locked order, sections used placeholder names that had to be reconciled later

AI Collaboration

- LLM used: Claude Sonnet 5.5
  - Important for entire group usability + less chat restrictions

Used for:

- Technical Articulation
- Rubric and Diagram Review

AI Collaboration

- Each member used their own chat and logged their responses using Discord Threads
- Prompting Strategy: gave context first (WezTerm docs, rubric, guidelines), asked for set formats such as tables, added missing info to fill in gaps in its knowledge
- Validation Strategy: checked claims against the official WezTerm docs, a different group member, other than the author, had to review any AI-assisted sections
- Contributed ~30% of the final report
- Most useful for explaining some terminology early-on and catching naming + logic inconsistencies in our diagrams
- Often sounded confident about details that were not included in our report

Thank You!

